Name:Khushi Singh
Class:D20C Roll no.34

Experiment No. 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:
- Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. Target Hash Generation with Nonce:
- Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. Proof-of-Work Puzzle Solving:
- Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. Merkle Tree Construction:
- Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program & Output

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

```
Enter a string: khushi
Hash: b6ed35e82cc571aa70dde3d57da6c7ed4cdd122be520c32c56c76fdb5566715d
```

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce
# Calculate the hash using SHA-256
```

```
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: khushi
Enter the nonce: 3
Hash Code: 440494122d853b1d7778d13c40128547af23b19b35ff5dadf13611687050eed4
```

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                        #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                              #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

```
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Store all levels of the tree, starting with the leaves
    tree = [transactions[:]]  # copy so we don't mutate the caller's list

    current_level = transactions[:]

    if len(current_level) == 1:
        return tree

    while len(current_level) > 1:
        if len(current_level) % 2 != 0:
            current_level.append(current_level[-1])  # duplicate last node if
odd

        next_level = []
        for i in range(0, len(current_level), 2):
            combined = current_level[i] + current_level[i + 1]
            hash_combined =
hashlib.sha512(combined.encode('utf-8')).hexdigest()
            next_level.append(hash_combined)

        tree.append(next_level)
        current_level = next_level

    return tree


def print_tree(tree):
    for level_num, level in enumerate(tree):
        label = "Leaves" if level_num == 0 else f"Level {level_num}"
        print(f"\n--- {label} ({len(level)} node{'s' if len(level) != 1 else
''}) ---")
        for i, node in enumerate(level):
            print(f"  [{i}] {node}")


# Example usage
transactions = ["Alice → Bob : $200", "Bob → Dave : $500", "Dave → Eve : $100",
"Transaction 4", "Transaction 5"]

merkle_tree = build_merkle_tree(transactions)
print_tree(merkle_tree)

print("\nMerkle Root:", merkle_tree[-1][0])
```

```
--- Leaves (5 nodes) ---
  [0] Alice → Bob : $200
  [1] Bob → Dave : $500
  [2] Dave → Eve : $100
  [3] Transaction 4
  [4] Transaction 5

--- Level 1 (4 nodes) ---
  [0] 644a505a42ed1cc13e15db1837163e12dc5aafd0333d715f28698f2f909da7afd962697d8b2306cc61d6bef69b548cc6586a89850144e8332db3fbdca98e2724
  [1] cb29a434857e32b3280785d9b7259323fd76eeb837bc8ebcc5a7619d00d0f52dd37ee00bf3e656a260f0280d09e15cee03bde496d525bb9ad8af0ad48ad9c343
  [2] 0a1e46546bee023cb27eecebeeff36160b44e2ead7569b8056a855e9b6a23d33dcfe5dd77d7509cbe0b2d7f735d86827ae8ef8a3aad602035fa44c8c31163f38
  [3] 0a1e46546bee023cb27eecebeeff36160b44e2ead7569b8056a855e9b6a23d33dcfe5dd77d7509cbe0b2d7f735d86827ae8ef8a3aad602035fa44c8c31163f38

--- Level 2 (2 nodes) ---
  [0] be9aeffefd306085527b88954ed81dd9effceba54756a2cc3e994c7f758916d4ec3ced8d1fe4f13e07b169205fde1ee2388fc6fb6b892ab04e726307cbd1ee9f
  [1] aa7fa5add527827dbd7cd788e777d3c194db40f12c72a94f024681fe3f0be5de3748cc670445339edbfd2b2663120a25767ba29d24d0ec26b94c190122caa7c4

--- Level 3 (1 node) ---
  [0] f538ad6ef3f6eabb984254b09ece46a7db22c9356def9c2ec92396ab3f8583123503b694d90d332af9d665ed2821f4e9969448d7cdb09c8540cb22cfd425ad77

Merkle Root: f538ad6ef3f6eabb984254b09ece46a7db22c9356def9c2ec92396ab3f8583123503b694d90d332af9d665ed2821f4e9969448d7cdb09c8540cb22cfd425ad77
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

· SHA-256 Hashing provides a secure and unique fingerprint for any input,ensuring data integrity.

· Nonce and Target Hashing allow the simulation of proof-of-work consensus,emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).

· Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.

· Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.